

Universidad Mayor

Facultad de Ciencias, Ingeniería y Tecnología
Escuela de Ingeniería Civil en Computación e Informática

Abstracta

Descubre, traduce, guarda y discute papers científicos en comunidad

INGT1003 - Arquitectura de Desarrollo (Móvil y Web)

Equipo de Trabajo:
Ricardo Cornejo
José Tejada
Nicolas Yevenes
Martín Fariña
Matías Quezada

Docente:
Carlos Muñoz S.

Fecha:
Octubre 2026
Santiago, Chile

Índice

1. Introducción **2**

2. Desarrollo **2**
2.1. Definición del Problema y Evidencia 2
2.2. Promesa de Valor 2
2.3. Identidad Visual y Coherencia de Marca 2
2.4. Arquitectura de Información 3
2.5. Arquitectura Cliente-Servidor y Capas 4
2.6. Selección de Lenguajes y Frameworks 5
2.7. Estimación de Costos (HH) y Mantención Operativa 5
2.8. Licencias y restricciones 6

3. Conclusión **7**

1 Introducción

El presente informe supone la primera propuesta arquitectónica y comercial de **Abstracta**, aplicación diseñada para facilitar el ciclo de descubrimiento, lectura y discusión de literatura científica. Actualmente, el ecosistema académico impone fricciones técnicas y lingüísticas que limitan el acceso al conocimiento de frontera. Este informe detalla el problema identificado, la promesa de valor, las licencias a utilizar, la arquitectura técnica de tres capas basada en el ecosistema de Cloudflare que dará soporte al producto, el detalle HH (horas-hombre) y el frontend que se aplicará en la aplicación.

2 Desarrollo

2.1 Definición del Problema y Evidencia

Leer ciencia en comunidad hoy está disperso y presenta barreras excluyentes. Para investigadores, estudiantes y población interesada en Chile, el ecosistema actual exige descubrir los artículos en un repositorio, gestionarlos en software de terceros, traducirlos en plataformas externas y discutirlos en redes no vinculadas al documento original, limitando así el alcance y discusión óptimas para la diseminación de conocimiento.

A lo anterior se suma la barrera del idioma, ya que la mayoría de los papers relevantes se publican en inglés. A la hora de diseñar un producto de software es conveniente enfocar este esfuerzo en tecnologías accesibles buscando así un diseño universal, lo que requiere considerar explícitamente las necesidades de los usuarios para quienes el inglés no es su lengua madre. Asumir un dominio nativo genera una brecha de accesibilidad severa; por ejemplo, según el EF English Proficiency Index (según un estudio del 2025) [3], Chile ocupa el puesto 54 de 123 países, reflejando un dominio ”moderado”(B2). Este nivel general suele ser insuficiente frente a la exigencia léxica requerida para comprender literatura científica compleja sin fricción cognitiva lo cual se define para un nivel C1/C2.

Adicionalmente, la dilución de las discusiones de trabajos académicos rompe la dinámica del trabajo colaborativo que enriquece el conocimiento. La evidencia demuestra que las redes de aprendizaje en línea se forman en torno a intereses comunes mediante la comunicación intermediada por computadora. Abstracta resuelve esto unificando el acceso y la comunicación en un solo entorno móvil.

2.2 Promesa de Valor

Para estudiantes e investigadores que sufren la barrera del idioma y la dispersión de herramientas al leer literatura científica, Abstracta les entrega un entorno móvil único para descubrir, traducir, leer sin conexión y discutir papers de arXiv con una comunidad académica constantemente actualizada.

2.3 Identidad Visual y Coherencia de Marca

Para formalizar los patrones de diseño, se estableció la visión de un producto que, acorde a la seriedad de los papers científicos, proponga un logotipo con estilo clásico y una paleta de colores enfocada en la lectura. A continuación, se detallan las tipografías, los colores y sus usos dentro de la interfaz:

- **Paleta base (Fondo y Texto):** Se optó por un fondo de lectura en tono pergamino claro (`#FBF8F0`), que busca reducir la fatiga visual para la lectura prolongada. Para el texto principal, se utiliza un gris carbón oscuro (`#211F1A`). Combinación seleccionada para suavizar así el contraste y acercarse a tonos cálidos.
- **Fuente de Logotipo y Títulos:** Se seleccionó la fuente **Lora** (Serif) en itálica. Su estilo es clásico e insinúa el rigor de las publicaciones académicas.
- **Fuente de Interfaz:** Para los textos funcionales y botones de los papers, se utiliza la fuente **IBM Plex Sans** (Sans-serif), la cual garantiza una alta legibilidad en espacios reducidos.
- **Colores de interacción:** Las acciones principales y botones de navegación destacan con un verde bosque (`#3F5D46`) que transmite concentración. Como color secundario, se aplica un rojo ladrillo (`#A13A2C`) para los indicadores de notificaciones y la acción de guardar en favoritos.

2.4 Arquitectura de Información

Abstracta se estructura mediante una barra de navegación principal que da acceso a cuatro módulos secundarios:

- **Home:** Feed personalizado de papers descubiertos vía la API de arXiv.
- **Social Feed:** Muro de debates, mensajeria, citas y comunidad.
- **Library:** Carpetas de guardado y lectura offline (PDF con guardado local).
- **Profile:** Ajustes de cuenta, idioma e intereses.

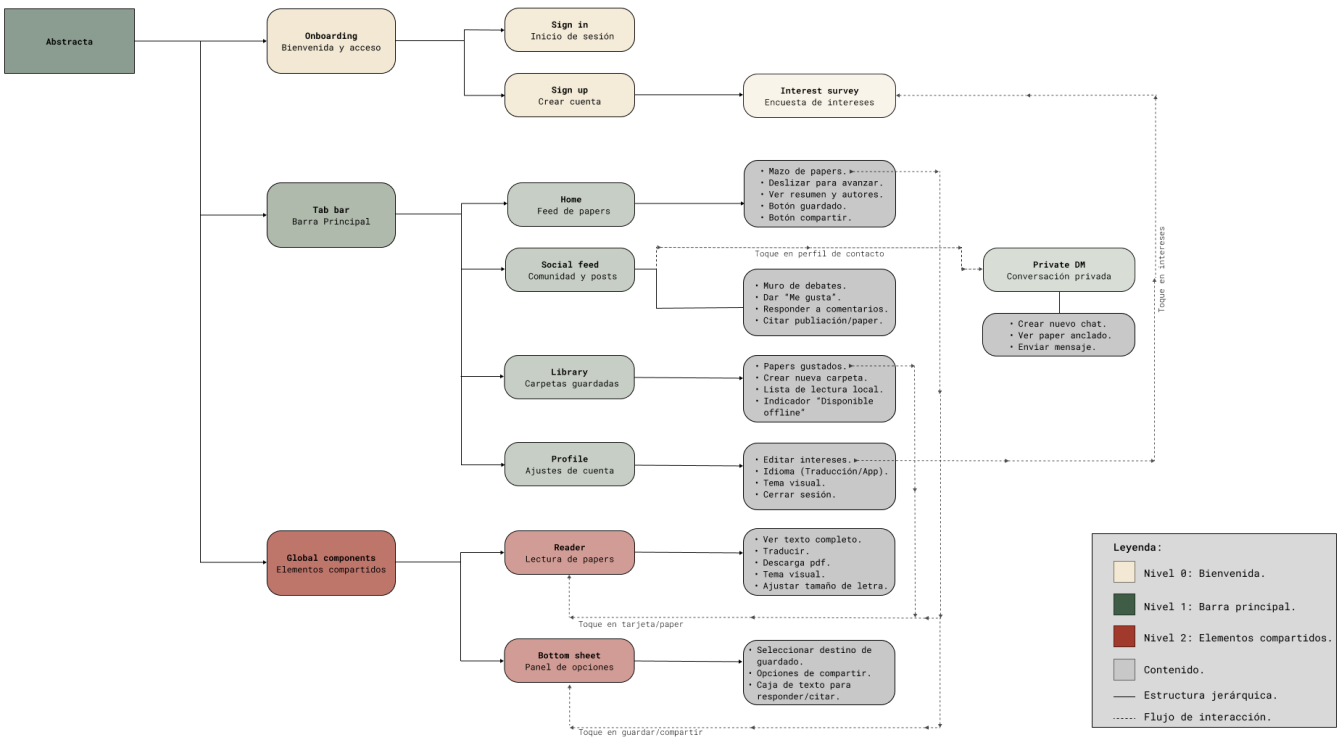


Figura 1: *Arquitectura de Información de Abstracta, ilustrando los flujos de interacción desde el Onboarding hasta los elementos globales compartidos.*

2.5 Arquitectura Cliente-Servidor y Capas

El sistema está diseñado utilizando una arquitectura distribuida de tres capas con responsabilidades estrictamente separadas:

1. **Capa 1 - Cliente (Navegador/PWA):** Desarrollada en Flutter/Dart. Concentra la interfaz, el estado de navegación local y la generación de PDFs. Solo se conecta a la API para peticiones de red necesarias, manteniendo un enfoque offline-first.
2. **Capa 2 - Funciones de Servidor (Serverless API):** Implementada en Cloudflare Pages Functions. Actúa como un middleware que expone una API RESTful mínima. Aquí reside la lógica para consultar la API de arXiv y orquestar Cloudflare Workers AI para traducciones.
3. **Capa 3 - Base de Datos:** Utiliza Cloudflare D1 (SQL) para persistir el estado compartido (perfiles, posteos, likes y metadatos de la biblioteca).

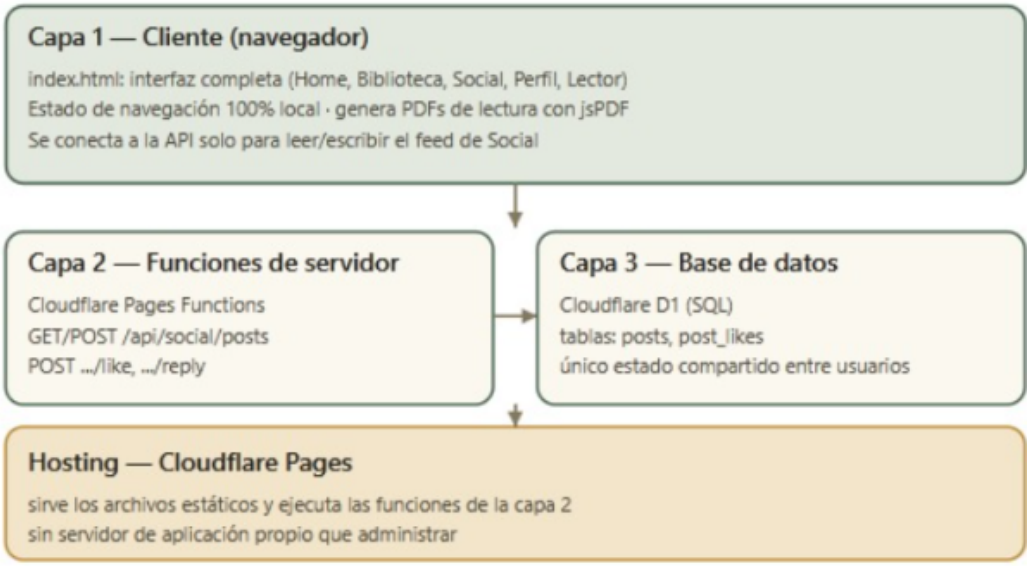


Figura 2: *Diagrama arquitectónico de tres capas y su flujo de interacción en Cloudflare.*

Recorrido de un caso de uso: Cuando un usuario solicita traducir un paper, el *Cliente (Capa 1)* hace una petición HTTP GET. La *Función de Servidor (Capa 2)* procesa la solicitud, verifica si la traducción ya existe en la *Base de Datos (Capa 3)* para ahorrar recursos. Si no existe, invoca a Workers AI, guarda el resultado en D1 y devuelve el JSON al cliente.

2.6 Selección de Lenguajes y Frameworks

Se evaluaron dos alternativas para el Frontend: **React Native** y **Flutter (Dart)**.

- **Criterios de evaluación:** Curva de aprendizaje, conocimientos previos, rendimiento multiplataforma, soporte para Progressive Web Apps (PWA) y ecosistema de renderizado de PDFs offline.
- **Decisión:** Se elige **Flutter con Dart**.
- **Argumento y costo asumido:** Flutter permite compilar a PWA y móvil nativo desde una sola base de código, facilitando el uso offline exigido por el producto. El **costo asumido** es que el tamaño del *bundle* web inicial en Flutter suele ser más pesado que en React, lo que requerirá estrategias agresivas de caché mediante Service Workers. El costo de infraestructura Back-end se mantiene en $0 mensuales utilizando las cuotas gratuitas de Cloudflare, hasta que se requiera migrar a opciones de pago en caso de un crecimiento en la demanda de la aplicación.

2.7 Estimación de Costos (HH) y Mantención Operativa

El cálculo de Horas Hombre (HH) para la construcción de *Abstracta* se fundamenta en un equipo de 4 integrantes trabajando durante un ciclo de desarrollo de 3 meses. Asumiendo una carga de dedicación de 10 horas semanales por persona, el proyecto requiere un total de **480 HH** para su lanzamiento, distribuidas secuencialmente según los hitos técnicos:

- **Hito 1 (Propuesta y Arquitectura) - 80 HH:** Comprende la definición del problema, arquitectura de información, decisión técnica justificada y el prototipado navegable.
- **Hito 2 (Desarrollo Back-end) - 160 HH:** Abarca el diseño del contrato de la API REST, modelado de la base de datos en Cloudflare D1, implementación lógica en el servidor y la separación de responsabilidades por capas.
- **Hito 3 (Producto Front-end e Integración) - 180 HH:** Incluye el consumo de la API en la interfaz móvil, manejo de estados, validaciones, soporte offline y persistencia de datos.
- **Documentación Final y Cierre - 60 HH:** Contempla pruebas de despliegue, elaboración del manual de usuario, análisis de tratamiento de datos (Ley 21.719) y la grabación del video promocional de la aplicación.

Costo de inversión inicial

Para ingresar al mercado actual se aplica una tarifa referencial para roles de arquitectura de soluciones e ingeniería en la nube de $15.625 CLP por hora (proporcional a una expectativa salarial de mercado de $2.500.000 CLP mensuales por jornada completa). Bajo esta métrica, la inversión integral de desarrollo asciende a $7.500.000 CLP.

Costo de mantención tras lanzamiento

Este valor de inversión humana se diferencia con el costo operativo de infraestructura, el cual logra mantenerse en $0 mensuales utilizando las cuotas gratuitas del ecosistema serverless de Cloudflare. Por lo tanto, el único costo de operación continuo es el tiempo humano para la mantención del sistema.

Para la etapa de post-lanzamiento, se estima una carga de mantención preventiva y correctiva (monitoreo de servicios, soporte a usuarios y resolución de errores) de **8 HH semanales** para el equipo (2 horas por integrante). Esto se traduce en:

- **Carga mensual de mantención:** 32 HH.
- **Costo operativo humano mensual:** $500.000 CLP.

Este modelo financiero demuestra que la viabilidad técnica y comercial exige un capital inicial destinado para su desarrollo ($7,5M CLP) y garantiza una operación a largo plazo de bajo costo operacional ($500.000 CLP mensuales).

2.8 Licencias y restricciones

Para el consumo de datos desde la API proporcionada por arXiv, el proyecto *Abstracta* se rige bajo estrictos lineamientos de interoperabilidad y propiedad intelectual. Es fundamental distinguir entre el uso de la interfaz de programación (API) y las licencias individuales de los documentos.

Licenciamiento de los Documentos

Para la distribución y adaptación de los artículos obtenidos mediante la API de arXiv, el proyecto se apoya en la licencia de uso abierto **CC BY 4.0**, la cual permite:

- **Compartir:** Copiar la obra y distribuir el material en cualquier medio o formato.
- **Adaptar:** Transformar y construir sobre la obra para cualquier propósito, incluso el comercial, siempre y cuando se le atribuya crédito al autor original.

En sintesis, los papers que se mostrarán serán los que el autor haya definido con la licencia CC BY 4.0.

Restricciones de arXiv

Para la utilización de su API, arXiv dispone de restricciones técnicas y legales como limitación de tasa, limitación de licencia, prohibición de almacenamiento en servidores propios, agradecimiento oficial y reglas de nombramiento de proyectos:

- **Limitación de tasa:** arXiv impone un límite de una solicitud HTTP cada tres segundos; en caso de ignorar la limitación, se bloqueará el acceso a las solicitudes.
- **Limitación de licencia:** arXiv proporciona los metadatos de las obras, pero estas cuentan con sus propias licencias de derechos de autor (como Creative Commons) proporcionadas por sus creadores.
- **Almacenamiento de obras en servidores propios:** arXiv prohíbe el almacenamiento de obras en servidores propios de los proyectos. Por esto, Abstracta únicamente proporcionará una interfaz para conectar los papers al dispositivo de los usuarios, donde podrán ser guardados en local sin alojar nunca una obra en nuestros servidores.
- **Agradecimiento oficial:** arXiv exige que se le realice un agradecimiento por el uso de su interoperabilidad y distribución de obras sin buscar un coste monetario.
- **Nombramiento de proyectos:** arXiv prohíbe que los proyectos que se realicen con su API tengan nombres que puedan asociar el producto a arXiv, además de prohibir representarlos como respaldados o apoyados por la institución.

3 Conclusión

A través de este primer hito, hemos logrado definir una propuesta de valor coherente y una base tecnológica sólida y real para Abstracta. Logramos estructurar una arquitectura de tres capas apoyada en Cloudflare que nos permite interoperar con la API de arXiv a costo cero. El desafío pendiente para el próximo hito será implementar rigurosamente la API RESTful en Cloudflare Functions y asegurar la correcta persistencia relacional en D1, garantizando que el motor responda fielmente a las necesidades del prototipo interactivo.

Bibliografía

[1] Cloudflare. (2026). *Cloudflare D1 and Pages Functions Documentation*. Recuperado de https://developers.cloudflare.com/

[2] Cornell University. (2026). *arXiv API Documentation*. Recuperado de https://info.arxiv.org/help/api/

[3] Education First. (2025). *EF English Proficiency Index*. Recuperado de https://www.ef.com/epi/

[4] Zaphiris, P., & Ang, C. S. (Eds.). (2009). *Human computer interaction: Concepts, methodologies, tools, and applications*. Information Science Reference.

Anexos

Anexo A: Declaración de uso de Inteligencia Artificial
Se utiliza la IA generativa Claude para la producción del prototipo y sus posteriores versiones de la aplicación.

Anexo B: Declaración de uso de Inteligencia Artificial para la estructura del informe
Se utiliza la IA generativa Gemini para la estructuración y apoyo en la redacción de este informe en Overleaf codificándose en LaTeX.

Anexo C: Enlaces oficiales del proyecto

- **Repositorio en GitHub:** `https://github.com/Abstracta-team/Abstracta`
- **Prototipo navegable (App):** `https://papers-65m.pages.dev/`